Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Program: Software Engineering

Operating Systems

Laboratory Work No. 2

Meet the OS You Will Build

Track B: Build Your Own Small OS (xv6)

Elaborated by:

st. gr. FAF-241
Luchiciov Alexei

Verified by:

Reiţman Patricia

Chișinău - 2026

Contents

1 Introduction **2**

2 Build and Boot xv6 **2**

2.1 Interesting Command Output 2

3 Using xv6 as Unix **3**

3.1 Observations 3

3.2 Interesting Command Output 3

4 Reading the Source Code **4**

4.1 Observations 4

4.2 Interesting Command Output 4

5 First xv6 Program: sleep **5**

5.1 Implementation 5

5.2 Build and Test 6

5.3 Interesting Command Output 6

6 Conclusion **7**

1. Introduction

The purpose of this laboratory work was to become familiar with **xv6**, a small Unix-like teaching operating system. During the laboratory work, xv6 was compiled from source code and started in the QEMU RISC-V emulator. I also used several commands inside xv6, inspected its source code, found system calls, and added my first user program.

The work was performed inside the Ubuntu virtual machine. The xv6 source tree was located in `~/xv6-riscv`. The project contained the main folders `kernel/` and `user/`, together with the `Makefile`, `README`, and filesystem image.

2. Build and Boot xv6

Before starting xv6, I checked the project directory and the installed QEMU emulator:

```
cd ~/xv6-riscv
ls
qemu-system-riscv64 --version
```

The directory contained:

```
LICENSE   README  kernel  test-xv6.py
Makefile  fs.img  mkfs    user
```

The installed emulator reported:

```
QEMU emulator version 10.2.1
```

Then xv6 was compiled and started using:

```
make qemu
```

After the build, the operating system booted successfully:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$
```

The `$` prompt showed that the xv6 shell was ready to accept commands.

2.1. Interesting Command Output

The boot output was interesting because it showed that the kernel started more than one RISC-V hart and then launched the initial shell process.

3. Using xv6 as Unix

Inside the xv6 shell, I used several standard Unix-like programs:

```
$ ls
$ cat README
$ echo hello xv6
$ ls | grep c
$ wc README
```

The `ls` command showed the programs included in the xv6 filesystem. Some of them were:

```
cat     echo     forktest     grep     init
kill    ln       ls           mkdir    rm
sh      stressfs usertests    wc       zombie
```

The echo command produced:

```
$ echo hello xv6
hello xv6
```

A pipe was also tested:

```
$ ls | grep c
cat            2 3 36728
echo           2 4 35592
wc             2 17 37728
sync           2 22 34944
console        3 23 0
```

The word count for `README` was:

```
$ wc README
48 336 2441 README
```

3.1. Observations

1. Three programs shipped with xv6 are `ls`, `cat`, and `echo`. Other programs such as `grep`, `wc`, and `mkdir` are also available.
2. For a pipe such as `ls | grep c` to work, the operating system needs to manage multiple processes and provide a communication mechanism between them. The output of one process is passed through a pipe and becomes input for another process.
3. The xv6 shell feels similar to the Linux shell from Lab 1 because it can run commands and use pipes, but it is much smaller and has fewer programs and features.

3.2. Interesting Command Output

The pipe command was the most interesting result in this part because it demonstrated that even this small operating system can connect two processes and transfer data between them.

4. Reading the Source Code

After leaving QEMU, I inspected the xv6 source from Ubuntu. The project is divided into `kernel/`, which contains the operating-system code, and `user/`, which contains normal programs.

I inspected the `ls` and `cat` programs with:

```
sed -n "1,40p" user/ls.c
sed -n "1,30p" user/cat.c
```

The main loop in `user/cat.c` uses `read()` and `write()`:

```
while ((n = read(fd, buf, sizeof(buf))) > 0) {
  if (write(1, buf, n) != n) {
    fprintf(2, "cat: write error\n");
    exit(1);
  }
}
```

I checked the system-call declarations in `user/user.h`. Relevant entries were:

```
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int open(const char *, int);
int pause(int);
```

Then I located the kernel implementations with:

```
grep -n "sys_read\|sys_write" kernel/sysfile.c | head
```

The result was:

```
69:sys_read(void)
83:sys_write(void)
```

4.1. Observations

1. `cat` uses `read()` to read data, `write()` to output data, `open()` and `close()` to work with files, and `exit()` to terminate.
2. In my xv6 copy, `sys_read` is implemented in `kernel/sysfile.c` starting at line **69**; `sys_write` starts at line 83.
3. Code in `kernel/` runs as the operating system, while `user/` contains ordinary programs that request services through system calls.

4.2. Interesting Command Output

The output below directly connected the user-level operations with their kernel implementations:

```
69:sys_read(void)
83:sys_write(void)
```

5. First xv6 Program: sleep

The final task was to create a new user program called `sleep`. The program takes one number from the command line, waits for that number of clock ticks, and then finishes.

5.1. Implementation

I created the file `user/sleep.c`. The final version used in my xv6 source tree was:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }

  pause(atoi(argv[1]));
  exit(0);
}
```

The program first checks that exactly one argument was provided. If the argument is missing, it prints the usage message and exits with an error code. Otherwise, `atoi()` converts the text argument into an integer and the program pauses for that number of ticks.

Version note. The laboratory sheet shows a call to `sleep()`, but the version of xv6 used in my VM declares `pause(int)` in `user/user.h`. My first build with `sleep()` produced an "implicit declaration of function 'sleep'" error. Replacing the call with `pause()` matched the installed xv6 source and allowed the program to compile.

The new program was then added to the `UPROGS` list in the `Makefile`:

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        $U/_forktest\
        $U/_grep\
        $U/_init\
        $U/_kill\
        $U/_ln\
        $U/_ls\
        $U/_sleep\
        $U/_mkdir\
        ...
```

This makes the build system compile the program and place it in the xv6 filesystem image.

5.2. Build and Test

After modifying the source and the `Makefile`, I rebuilt the operating system:

```
make clean
make qemu
```

During the successful build, the new program was compiled and linked:

```
riscv64-linux-gnu-gcc ... -c -o user/sleep.o user/sleep.c
riscv64-linux-gnu-ld ... -o user/_sleep ...
```

After xv6 booted again, `ls` showed the new program:

```
ls             2 10 42880
sleep          2 11 35280
mkdir          2 12 35568
```

The normal test was:

```
$ sleep 10
$
```

The shell returned after a short pause, so the program worked correctly.

I also tested the error case by running the command without an argument:

```
$ sleep
usage: sleep <ticks>
$
```

This confirmed that the argument check and usage message worked as expected.

5.3. Interesting Command Output

The most important result of the laboratory work was seeing `sleep` in the xv6 filesystem and successfully running it:

```
sleep          2 11 35280
$ sleep 10
$
```

This was the first program I added to an operating system that I also compiled and booted myself.

6. Conclusion

In this laboratory work I compiled and booted xv6 on the RISC-V QEMU emulator and used its small Unix-like shell. Commands such as `ls`, `cat`, `echo`, `grep`, and `wc` showed that xv6 provides familiar Unix concepts even though it is much smaller than Linux.

I also inspected the relationship between user programs and the kernel. The `cat` program uses system calls such as `read()` and `write()`, while their kernel-side implementations can be found in `kernel/sysfile.c`. This made the boundary between `user/` and `kernel/` more understandable.

Finally, I created the `sleep` user program, added it to the xv6 build, rebuilt the operating system, and tested it successfully. In the current xv6 version used on my VM, the waiting system call is named `pause()`, so the program calls `pause(atoi(argv[1]))`. This laboratory work was my first practical experience with compiling an operating system and adding a program that runs inside it.